NEA - Group Creator for School Projects

Amelie Machin

Contents

1 Analysis **1**
1.1 Problem statement 1
1.2 Research 2
1.2.1 Sources of inspiration 2
1.2.2 Algorithm 3
1.2.3 Group A skills 3
1.2.4 Inputting data 4
1.2.5 Accounts 4
1.2.6 Internal data storage 4
1.2.7 External data storage 4
1.2.8 GUI library 4
1.3 Dialogue 5
1.4 Objectives 6
1.5 Modelling 6

1 Analysis

1.1 Problem statement

To form the perfect grouping for a group project within a school, many factors need to be taken into consideration to prevent students from feeling incapable of completing the project. These factors include skills of the students, skills required by the project, and number of students in the group.

This project aims to create a tool to aid teachers in ensuring the students are separated into groups where the students are best able to get along and combine their individual skills to achieve their project's aims. The user can input data about student relations and skills, data about the projects' skill requirements and group sizes and the program will output an optimal set of groups in the form of a table on a GUI to present the results in a readable way and an option to export the data as a CSV.

This program will not find, assume, or generate information about the students or project, it will only work with the data inputted to assign a group of students to a project.

Students will also benefit from this tool as they would be able to apply their skills to a project in a group of people they can enjoy time with, rather than being stuck on a project they have no interest in with people they do not get along with.

1.2 Research

1.2.1 Sources of inspiration

1. Random group generator from ClassTools.net: This tool is used frequently

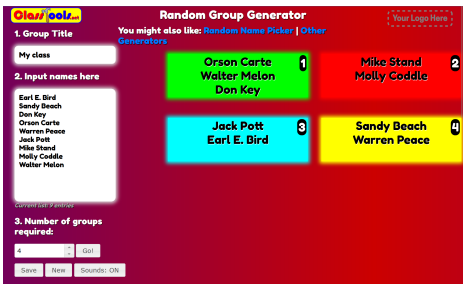


Figure 1: ClassTools's random group generator tool with inputs and outputs visible on the same page

   by my teachers to create random groups for group activities. The GUI is very simple to use as there is little data that needs to be entered, so can all fit down the left of the screen. The output is very easy to read as each group is visually separated using both coloured boxes and numbering. However, this tool is too simple because it lacks any way to group with conditions, which is part of my scope, as they are purely randomised. This means that teams could quit often be unfairly balanced in terms of skill, or group students together that are known to not work well together. The teacher's only option to solve this is to re-roll until the output is suitable enough for the task manually. This gave me the idea for the algorithm to be the simulation of this re-rolling and selection of optimal outputs.

2. keamk: This tool creates groups that all have a balanced level of skill. I like how the skill level is inputted as a rating between 1-5 as this is intuitive for the user. I also like the extra details that are outputted, such as how many are in each group next to the group name and the skill levels next to each individual. Furthermore, the option to go back, make small changes to the data inputted and get new groupings was very quick and useful. However, this tool can only create balanced groups, not groups with different objectives as is in my scope, and there is no way to assign a certain number of participants to each group, as the program puts the same number (or one away) of participants in each group.

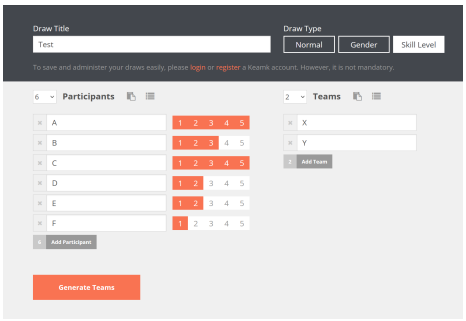


Figure 2: Keamk's menu for inputting data



Figure 3: Keamk's outputs

1.2.2 Algorithm

I have researched into representing students and projects into bipartite graphs, and then using matching algorithms such as the alternate path algorithm to join students to projects. However, this will only match one student to one project, not groups so is not suitable. I then researched applying the algorithm repeatedly to graphs with randomised students, but this means that some students will never be joined into a group purely based on chance and that the groups formed are such that each individual is best for the project, not that the overall group is best suited for the project, so teamwork will not as high as it could be. I then researched using a grouping algorithm, such as explained here to create similar groups then using bipartite matching algorithms to match them. However this only creates groups with similar skill sets, not skill sets that very based on the project.
I am currently looking into a genetic algorithm where each individual is made up of a possible solution and the fitness score is a calculation between how suited each group is to the assigned project. The solutions with lower fitness scores are discarded and the higher ones crossover to form new solutions. Some of these solutions then have random mutations, which involve swapping of students between projects. New fitness scores are calculated and the process repeats a predetermined number of times. Also simulated annealing. maybe.

1.2.3 Group A skills

data structures, OOP

1.2.4 Inputting data

For the inputting of data, I have decided that it can be inputted through txt and csv files (excel files??), through databases, manually on the GUI, or any combination of those methods. This is because the data for different individuals and different projects could be stored in different ways within the same system, and manual tweaks may be needed by the user to ensure all data is as is needed by the application. My third party...

1.2.5 Accounts

I have decided against implementing an account system because I don't think it will provide any value to the program. Accounts are typically used for ensuring data can only be accessed by the users with the correct permissions. As this program will be designed to be run locally, all the data will be stored and only accessible on the machine the program was run on, so the OS can handle user security.

1.2.6 Internal data storage

I chose to store the data about the students and projects in their own classes, that can be accessed from within a VYfivbeivuVBI

1.2.7 External data storage

I have researched into whether information about projects and students should be stored somewhere, and if so, how. I have concluded that they should be stored in relational databases that are accessible and modifiable using parametrised SQL.

I chose to store the data because being able to make minor changes to the data and then re-running the program quickly is a very valuable feature that could save users' time and energy as they would not have to go through the entire process of entering every piece of data just for small changes to some of the data. This feature is present in keamk's grouping tool, and I found it very useful in my testing.

I chose databases to store the data because I have more experience working with relational databases and SQL then any other form of data storage, such as in files. The data is structured so databases would allow for efficient access and modification of specific entries.

1.2.8 GUI library

Between the two main python GUI libraries, pygame and Tkinter, I chose Tkinter because of the larger library of classes of pre-made, customisable widgets that can be used more easily to produce a simple, easy-to-use tool, such as a dialog that allows users to graphically navigate through their files to find their data file [Fig.4]. On the other hand, with my prior experience with pygame, I

have found it to be more suited towards building games and graphically-complex tools, which is not what I am aiming for in this project. I am aiming to have a GUI that is quick and simple to create and edit, and has user-friendly features for inputting, editing, and reading data. I also have more experience with Tkinter, which will reduce the amount of time I spend on the GUI and allow me to focus on developing and improving the grouping algorithm.

1.3 Dialogue

My third party is Dr Ed Horncastle, the lead teacher for the Exeter Mathematics Certificate (EMC) at my school, which is where groups of 3 or 4 students work together on research projects set by University academics or companies. He is suitable because he has to make these groups based on the skills of the students and match them to projects every year, so he has a thorough understanding of how this tool could work and would benefit from using this tool. I asked him the following questions:

1. What is the current process for creating EMC groups?
2. How is the data about the students and projects currently stored?
3. What other metrics are taken into consideration when deciding groups, other than student relations and project skill requirements?
4. q(How to input the data?)
5. How would you prefer to input a project's skill requirements? For example, just noting that a skill would be involved, or a numerical scoring system of how necessary that skill is.
6. How would you prefer to input a student's skill set?
7. Would it be worthwhile to factor in student relations into my algorithm?

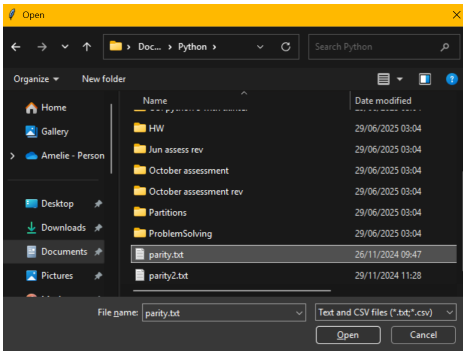


Figure 4: Tkinter's file dialog, configured to only show .txt and .csv files

8. If so, how would be best to input this data? For example, a list of students that should/should not be grouped together.

9. Are there any other features that would be useful for this tool to include?

1.4 Objectives

The projects should be saved to a database that can be edited graphically and check boxes to select which projects are a part of that run.

1. (Entering the data)

2. (processing the data)

3. (outputting the data)

1.5 Modelling

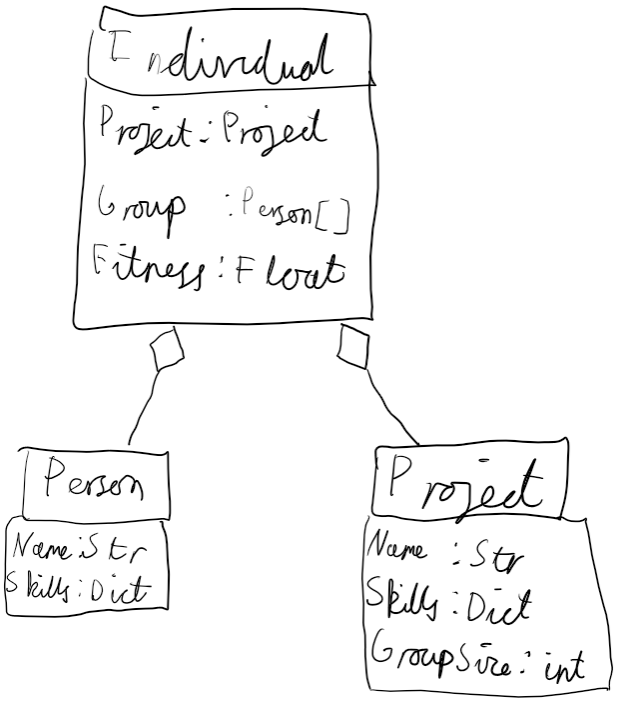


Figure 5: Class diagram showing relationship between the classes within the program and their attributes